

西安交通大学
XI'AN JIAOTONG UNIVERSITY

人工智能
Artificial Intelligence

- 主讲：刘欢
- 西安交通大学计算机学院
- E-mail：huanliu@xjtu.edu.cn

第五章 搜索策略

- 5.1 基本概念
- 5.2 状态空间的搜索策略
- 5.3 与/或树的搜索策略

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.1 基本概念

5.1.1 什么是搜索

- 采用某种策略，在知识库中寻找可利用的知识，从而构造一条代价较小的推理路线，使问题得到解决的过程称为搜索。
- 搜索分为盲目搜索和启发式搜索。
  - 盲目搜索是按照预定的控制策略进行搜索，在搜索过程中获得的中间信息不用来改进控制策略。
  - 启发式搜索是在搜索中加入了与问题有关的启发性信息，用以指导搜索朝着最有希望的方向前进，加速问题的求解过程并找到最优解。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.1.2 状态空间表示法

（1）很多问题的求解过程都可以看作是一个搜索过程。问题及其求解过程可以用状态空间表示法来表示。

（2）状态空间用“状态”和“算符”来表示问题。

- 状态：状态用以描述问题在求解过程中不同时刻的状态，一般用一个向量表示：

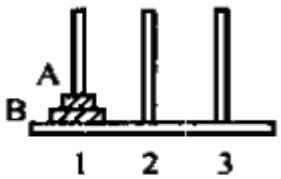
$$S_K = (S_{k0}, S_{k1} \ldots)$$

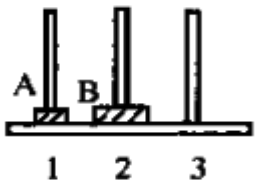
- 算符：使问题从一个状态转变为另一个状态的操作称为算符。在产生式系统中，一条产生式规则就是一个算符。
- 状态空间：由所有可能出现的状态及一切可用算符所构成的集合称为问题的状态空间。

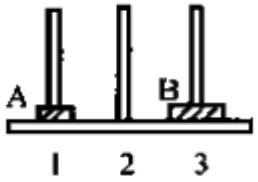
西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.1.2 状态空间表示法

- 例5.1 二阶梵塔问题。设有三根钢针，在1号钢针上穿有A、B两个金片，A小于B，A位于B的上面。要求把这两个金片全部移到另一根钢针上；而且规定每次只能移动一片，任何时刻都不能使B位于A的上面。











5.1.2 状态空间表示法

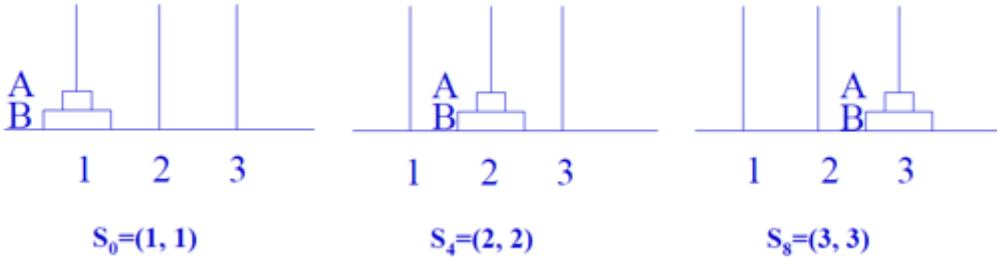
（1）设用$S_K = (S_{k0}, S_{k1})$表示问题的状态（第k个状态），S_{k0}表示金片A所在的钢针号，S_{k1}表示金片B所在的钢针柱号，全部可能的状态有九种：

$$S_0 = (1,1), \quad S_1 = (1,2), \quad S_2 = (1,3)$$

$$S_3 = (2,1), \quad S_4 = (2,2), \quad S_5 = (2,3)$$

$$S_6 = (3,1), \quad S_7 = (3,2), \quad S_8 = (3,3)$$

问题的初始状态集合为$S = \{S_0\}$,目标状态集合为$G = \{S_4, S_8\}$。



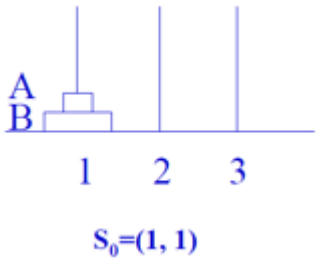
5.1.2 状态空间表示法

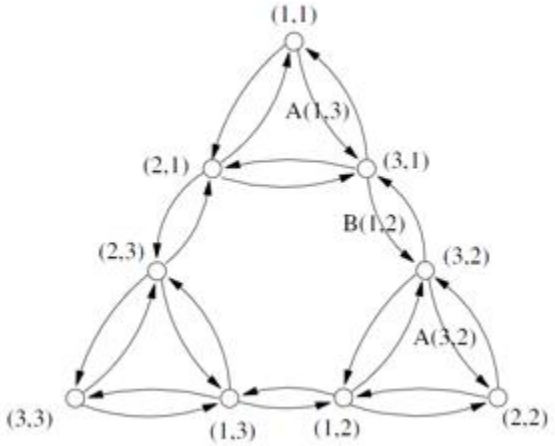
（2）算符：$A(i,j)$及$B(i,j)$。$A(i,j)$表示把A金片从第i号钢针移到第j号钢针。$B(i,j)$与之同理。算符共有12个：

$$A(1,2)\ A(1,3)\ A(2,1)\ A(2,3)\ A(3,1)\ A(3,2)$$

$$B(1,2)\ B(1,3)\ B(2,1)\ B(2,3)\ B(3,1)\ B(3,2)$$

- 根据上述9种可能的状态和12种操作，可构成二阶梵塔问题的状态空间图。

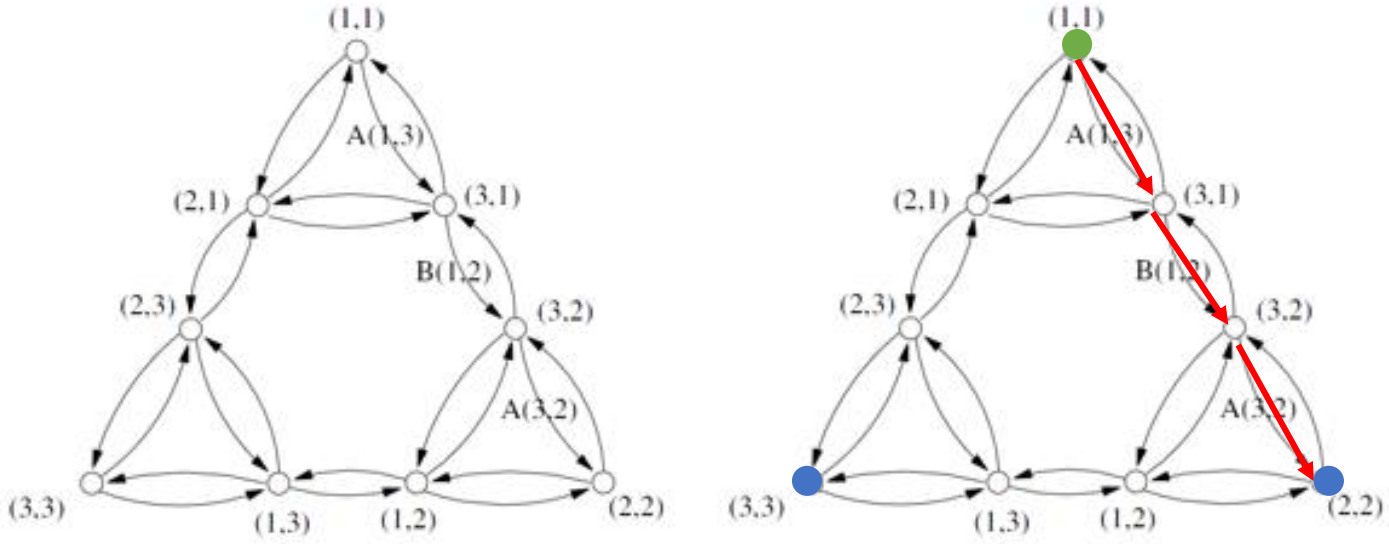




5.1.2 状态空间表示法

（3）在状态空间图中，从初始节点(1,1)到目标节点(2,2)或(3,3)的任何一条通路都是问题的一个解。

- 其中最短的路径长度是3，它由3个算符组成。
- 例如：从(1,1)开始，通过使用算符A(1,3),B(1,2),A(3,2)，可到到(2,2)。



5.1.2 状态空间表示法

- 采用状态空间表示方法，首先要把问题的一切状态都表示出来，其次要定义一组算符。

- 问题的求解过程是一个不断把算符作用于状态的过程。如果在使用某个算符后得到的新状态是目标状态，就得到了问题的一个解。这个解就是从初始状态到目标状态所采用算符的序列。使用算符最少的解称为最优解。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.1.3 与/或树表示法

- 与/或树是用于表示问题及其求解过程的另一种方法，通常用于表示比较复杂问题的求解。
- “与”树(AND TREE)：一棵树有非常多的分支，如果这些分支都是“与”(AND)关系，也就是都必须要同时满足，那么就可以称之为“与”树(AND TREE)。（必修课）
- 把大象放入冰箱需要三步：

  1）打开冰箱

  2）放入大象

  3）关上冰箱

这三步无论少了哪一步，都不能完全完成将大象放入冰箱的动作，所以必须全部完成，这三步操作就是“与”关系。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.1.3 与/或树表示法

- 与/或树是用于表示问题及其求解过程的另一种方法，通常用于表示比较复杂问题的求解。
- “或”树(OR TREE)：一棵树有非常多的分支，每个分支只需要满足其中一项即可，那么就可以称之为“或”树(OR TREE)。（选修课）
- 肚子饿了吃点什么：

  1）吃面

  2）吃饭

  3）吃水饺

很显然，我们只需要其中一样就可以解决我们肚子饿的问题，这三个选择就是“或”的关系。

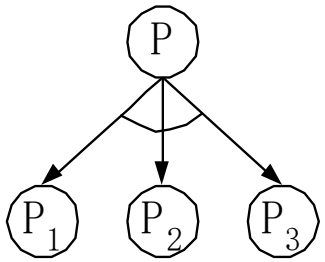
5.1.3 与/或树表示法

- 对于一个复杂问题，直接求解往往比较困难。此时可通过下述方法进行简化：

➢ 分解

✓ 把一个复杂问题分解为若干个较为简单的子问题，每个子问题又可继续分解。重复此过程，直到不需要再分解或者不能再分解为止。如此形成“与”树。

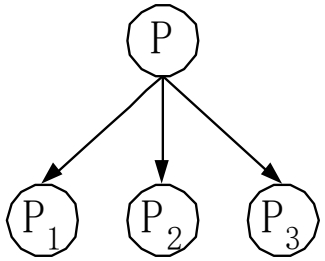
将一个问题分解为n个问题，这n个问题都必须解决。



与树

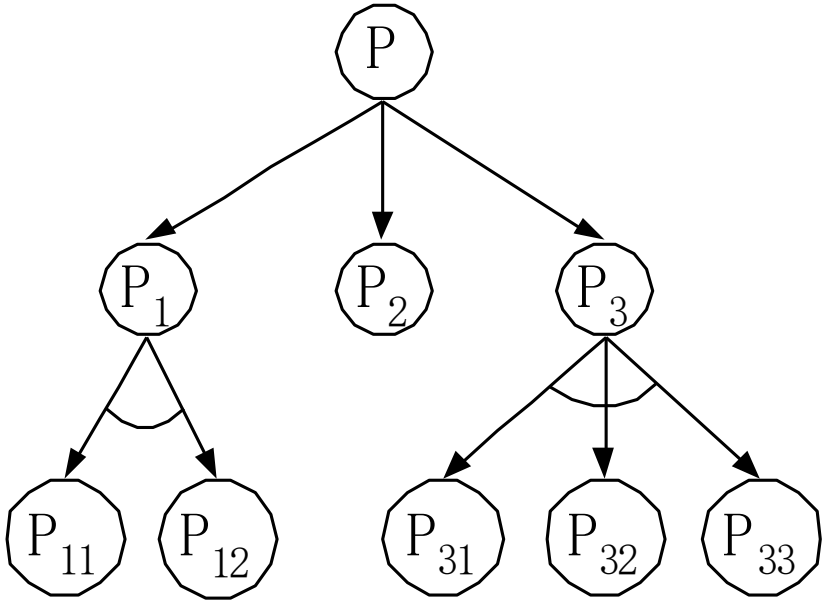
➢ 等价变换

✓ 利用同构或同态的等价变换，把原问题变换为若干个较为容易求解的新问题。如此形成“或”树。将一个问题转换为n个问题，解决n个问题之一就可以。

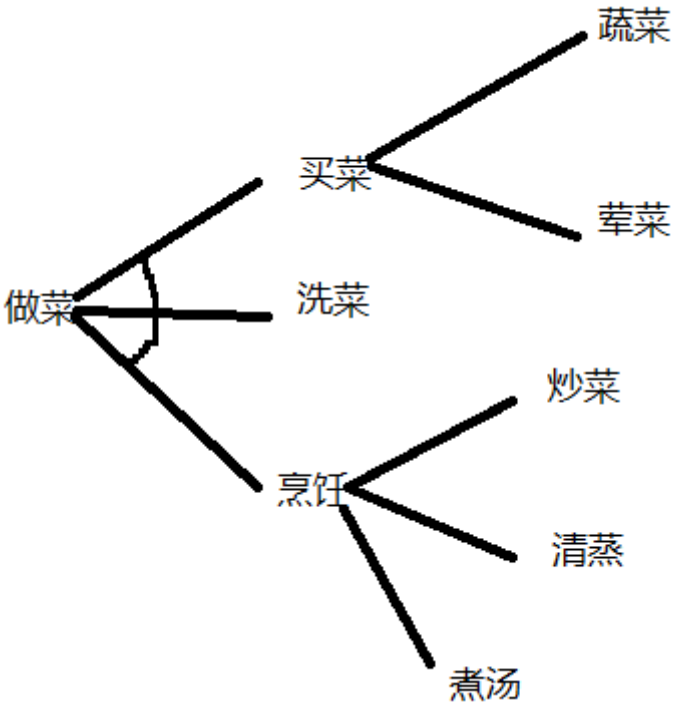


或树

与/或树



与/或树



西安交通大学 XI'AN JIAOTONG UNIVERSITY

一些基本概念

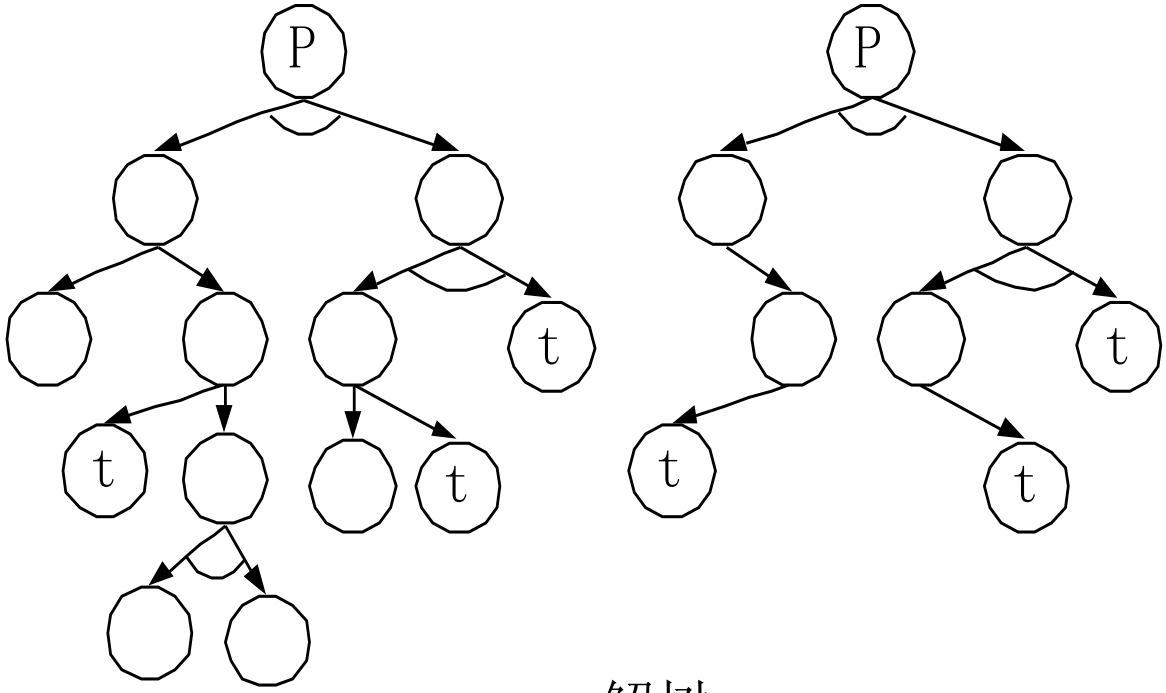
- 本原问题
  - 不能再分解或变换，而且直接可解的子问题。
- 端节点与终止节点
  - 在与/或树中，没有子节点的节点统称为端节点；本原问题所对应的节点称为终止节点。
- 可解节点
  - 在与/或树中，满足下列条件之一者，称为可解节点：
    - 它是一个终止节点；
    - 它是一个“或”节点，且其子节点中至少有一个是可解节点；
    - 它是一个“与”节点，且其子节点全部是可解节点。
- 不可解节点
  - 关于可解节点的三个条件全部不满足的节点。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

一些基本概念

- 解树

  由可解节点所构成，并且由这些可解节点可推出初始节点为可解节点的子树称为解树(t表示终止节点)。



解树

三阶梵塔问题的与/或树



- 若用(i,j,k)表示C在柱子i，B在柱子j，A在柱子k上。根据与/或树方法，问题可分解为以下三个子问题：

（1）把A和B移动到2号钢柱上的双金片移动问题。即：

$$(1,1,1) \longrightarrow (1,2,2)$$

（2）把C移动到3号钢柱上的单金片移动问题。即：

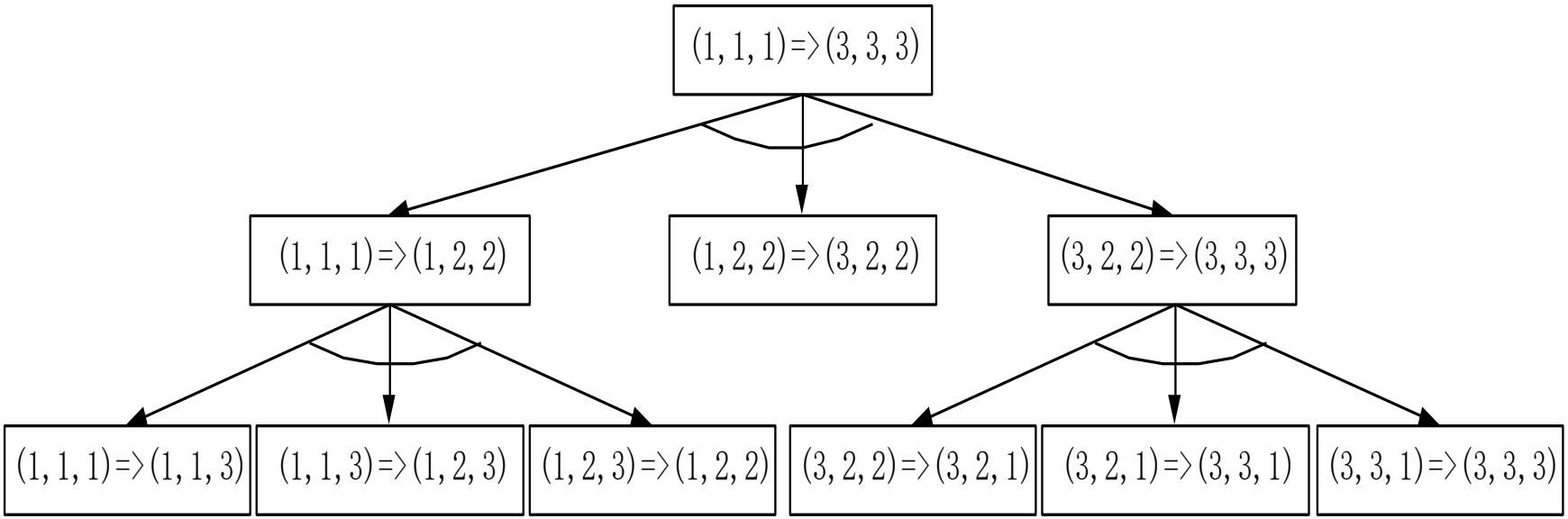
$$(1,2,2) \longrightarrow (3,2,2)$$

（3）把A和B移动到3号钢柱上的双金片移动问题。即：

$$(3,2,2) \longrightarrow (3,3,3)$$

- 其中，子问题（1）和（3）都是二阶梵塔问题，还可以再分解，（2）是本原问题。

三阶梵塔问题的与/或树



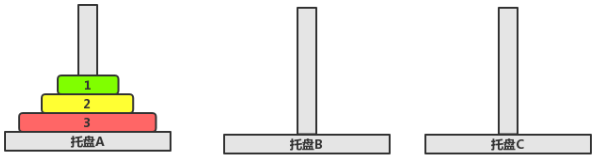
- 在该与/或树中，有7个终止节点，它们分别对应着7个本原问题。如果把这些本原问题从左至右排列起来，即得到了原始问题的解：

  (1,1,1) → (1,1,3)、(1,1,3) → (1,2,3)、(1,2,3) → (1,2,2)、(1,2,2) → (3,2,2)、(3,2,2) → (3,2,1)、(3,2,1) → (3,3,1)、(3,3,1) → (3,3,3)

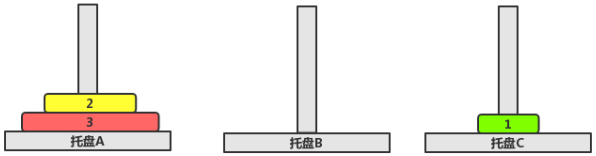
西安交通大学 XI'AN JIAOTONG UNIVERSITY

汉诺塔算法图解

准备：以三个盘子（1、2、3）为例，借助托盘B从托盘A上的盘子移动至托盘C

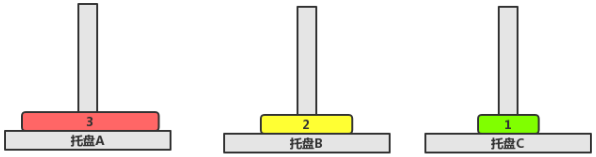


第一步：将盘子1从托盘A移动至托盘C



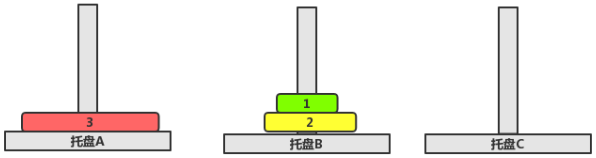
$(1,1,1) \rightarrow (1,1,3)$

第二步：将盘子2从托盘A移动至托盘B



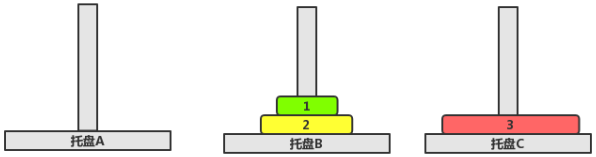
$(1,1,3) \rightarrow (1,2,3)$

第三步：将盘子1从托盘C移动至托盘B



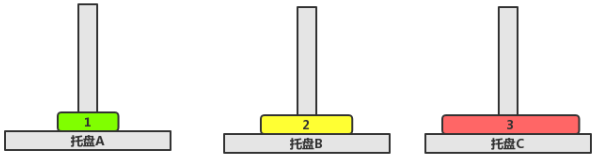
$(1,2,3) \rightarrow (1,2,2)$

第四步：将盘子3从托盘A移动至托盘C



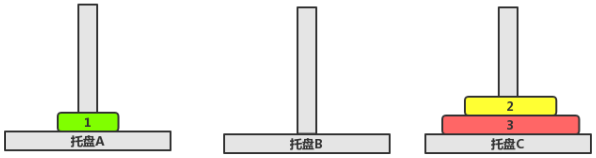
$(1,2,2) \rightarrow (3,2,2)$

第五步：将盘子1从托盘B移动至托盘A



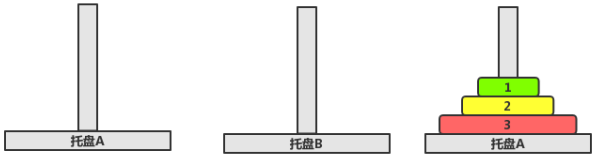
$(3,2,2) \rightarrow (3,2,1)$

第六步：将盘子2从托盘B移动至托盘C



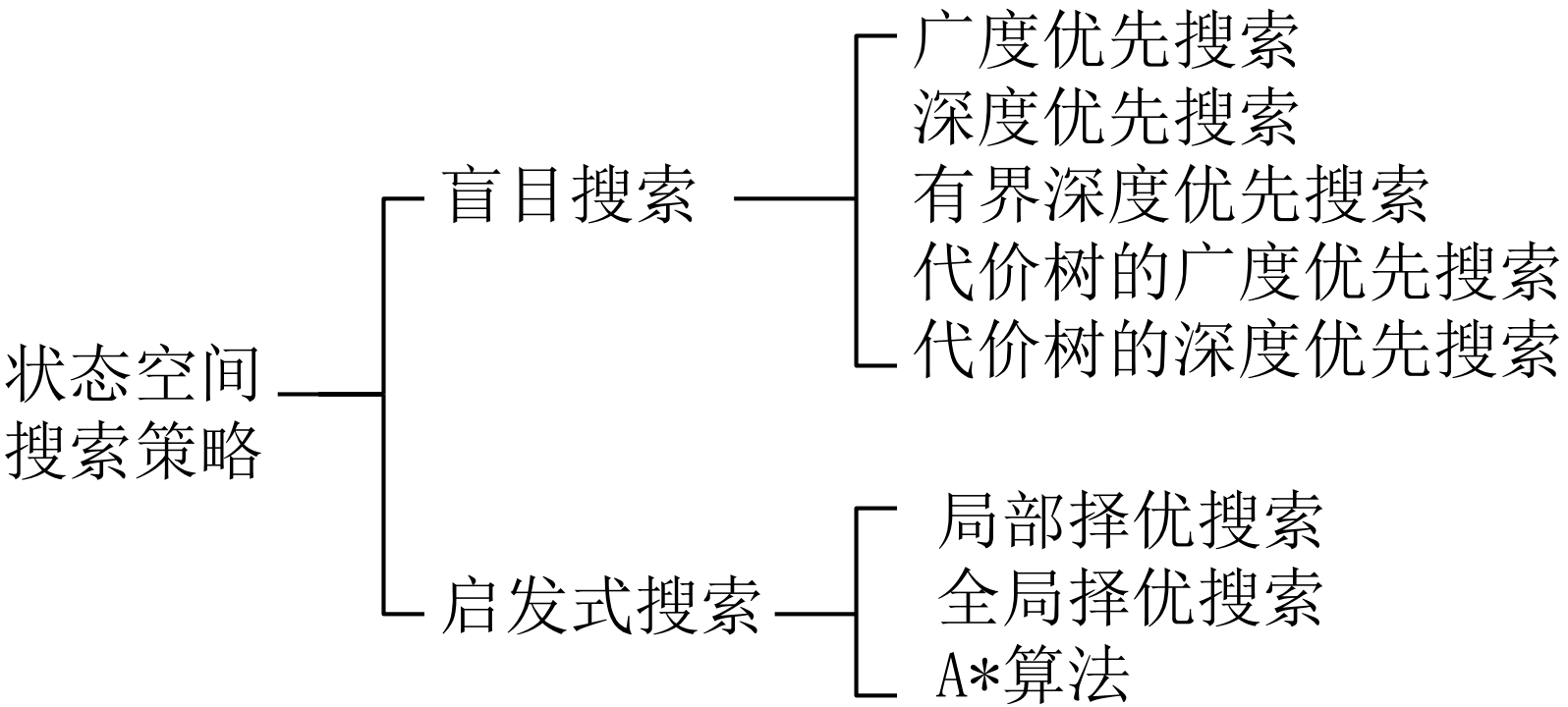
$(3,2,1) \rightarrow (3,3,1)$

第七步：将盘子1从托盘A移动至托盘C



$(3,3,1) \rightarrow (3,3,3)$

5.2 状态空间的搜索策略



5.2 状态空间的搜索策略

- 盲目搜索的特点：
  - 搜索按规定的路线进行，不使用与问题有关的启发性信息；
  - 适用于其状态空间图是树状结构的一类问题。（同样适用于一般的图）
  - 具有盲目性，效率较低，容易出现“组合爆炸”问题。
- 启发式搜索的特点：
  - 启发式搜索要使用与问题有关的启发性信息，并以这些启发性信息指导搜索过程，可以高效地求解结构复杂的问题。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.1 状态空间的一般搜索过程

- 状态空间搜索是一种用于解决问题的算法，它通过在状态空间中寻找解决问题的路径来达到目标状态。
- 搜索过程中将状态一个一个地加入OPEN表，然后检查它们是否是目标状态，如果不是，则将其邻居状态加入OPEN表中。在搜索过程中，将已经被处理的状态加入CLOSED表，以避免重复搜索相同的状态。
- OPEN表和CLOSED表的作用是对搜索过程中的状态进行管理和记录，以便能够高效地找到目标状态。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.1 状态空间的一般搜索过程

OPEN表和CLOSE表

- OPEN表用于存放刚生成的节点。对于不同的搜索策略，节点在OPEN表中的排列顺序是不同的。（未被探索）
- CLOSE表用于存放已扩展的节点。对一个节点的扩展是指：用所有可适用的算符对该节点进行操作，生成一组子节点。（已被探索）

OPEN表

状态节点	父节点

CLOSE表

编号	状态节点	父节点

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.1 状态空间的一般搜索过程

1. 把初始节点S_0放入OPEN表，并建立目前只包含S_0的图，记为G；
2. 检查OPEN表是否为空，若为空则问题无解，退出；
3. 把OPEN表的第一个节点取出放入CLOSE表，并记该节点为n；
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出；
5. 扩展节点n，生成一组子节点。把其中不是节点n先辈的那些子节点记做集合M，并把这些子节点作为节点n的子节点加入G中；

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.1 状态空间的一般搜索过程

6. 针对M中子节点的不同情况，分别进行如下处理：（动态更新open表及图G）
   ① 对于那些未曾在G中出现过的M成员，设置一个指向父节点（即节点n）的指针，并把它们放入OPEN表；（子节点不在OPEN表）
   ② 对于那些先前已经在G中出现过的M成员，确定是否需要修改它指向父节点的指针；（子节点在OPEN表中）
   ③ 对于那些先前已在G中出现并且已经扩展了的M成员，确定是否需要修改其后继节点指向父节点的指针；（子节点在CLOSE表中）
7. 按某种搜索策略对OPEN表中的节点进行排序；
8. 转第2步。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

一些说明

- 一个节点经一个算符操作后只生成一个子节点。但适用于一个节点的算符可能有多个，此时就会生成一组子节点。这些子节点中可能有些是当前扩展节点的父节点、祖父节点等，此时不能把这些先辈节点作为当前扩展节点的子节点。

（第5步：把其中不是节点n先辈的那些子节点记做集合M）

西安交通大学 XI'AN JIAOTONG UNIVERSITY

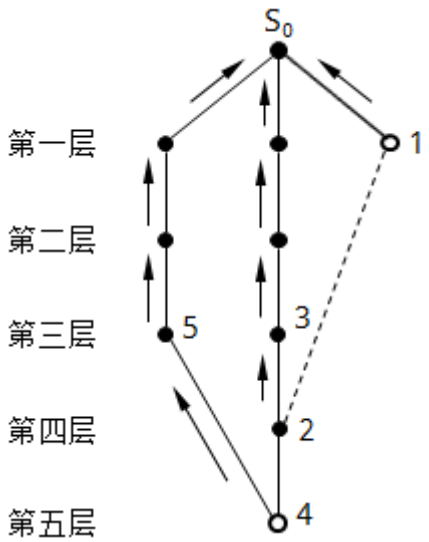
一些说明

- 一个新生成的节点，它可能是第一次被生成的节点，也可能是先前已作为其它节点的子节点被生成过，当前又作为另一个节点的子节点被再次生成。此时，它究竟应选择哪个节点作为父节点？一般由初始节点到该节点的代价来决定，处于代价小的路途上的那个节点就作为该节点的父节点。

（第6步之②：对于那些先前已经在G中出现过的M成员，确定是否需要修改它指向父节点的指针）

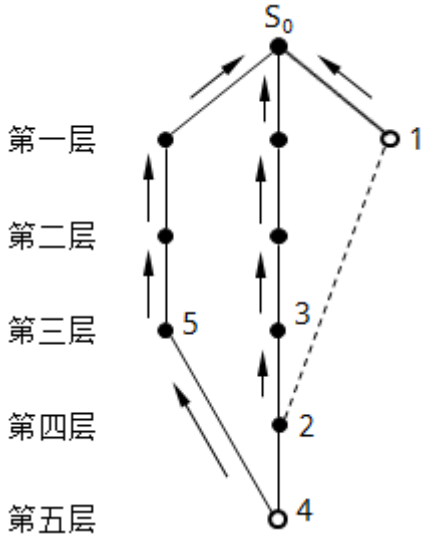
西安交通大学 XI'AN JIAOTONG UNIVERSITY

一些说明

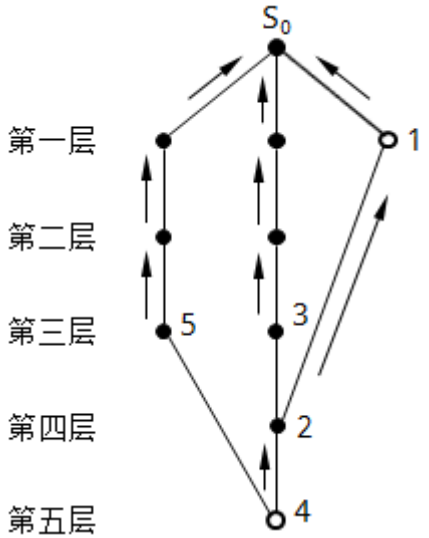


- 现举例说明，设左图为搜索过程所形成的图，其中实心黑点代表已扩展了的结点，它们位于CLOSED表上；空心圆圈代表未扩展的结点，它们位于OPEN表上，有向箭头是指向父结点的指针，它们是在步骤6形成的，例如结点3是结点2的父结点。

一些说明

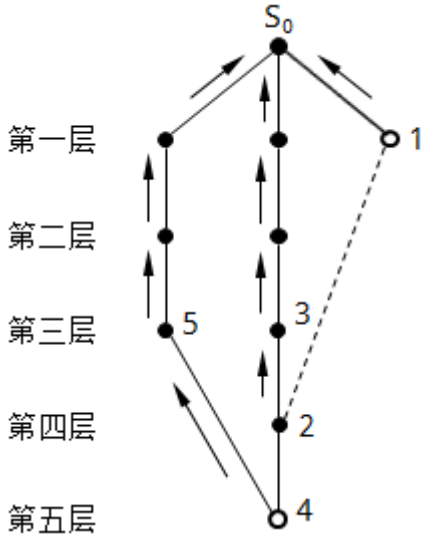


- 假设现在要扩展结点1，并且只生成单一的子结点2。
  - ✓ 但是目前结点2已有父结点3，那结点2在先前扩展结点3时已被生成了，现在又作为结点1的子结点被再次生成。此时，为确定哪一个结点作为结点2的父结点，需要计算路径代价。

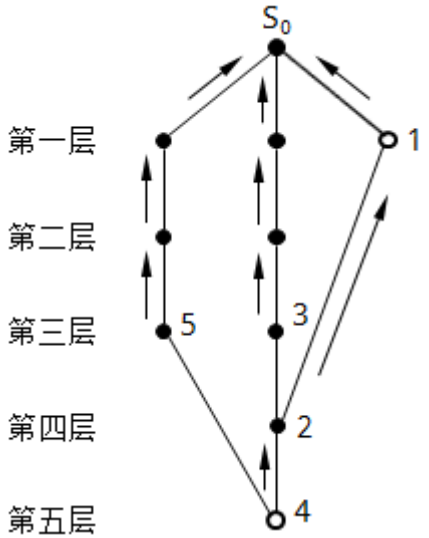


- 假设每条边的代价为1，则从S_0经右边路径到结点2的代价为2，而从S_0经中间路径到结点2的代价为4，显然右边路径到结点2的代价较小，因此应修改结点2指向父结点的指针让它指向结点1，即把结点1作为结点2的父结点，不再以结点3作为它的父结点。

一些说明



- 另外，结点4既是结点2的子结点又是结点5的子结点。当结点2以结点3作为父结点时，由于从S_0经中间路径到结点4的代价大于从S_0经左边路径到结点4的代价，所以结点4以结点5为父结点。



- 但是，经扩展结点1之后，从S_0经右边路径到结点4的代价为3，而从S_0经左边路径到结点4的代价为4，所以结点4不能再以结点5为父结点，而需要改为以结点2为父结点。此时，搜索图如下图所示。这就是搜索过程步骤6所阅述的内容。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

一些说明

- 通过搜索所得到的图称为搜索图。由搜索图中所有的节点及反向指针（指向父节点的指针）所构成的集合是一棵树，称为搜索树。
- 在搜索过程中，一旦某个被考察的节点是目标节点，就得到了一个解。该解是由从初始节点到该目标节点的路径上的算符构成。
- 如果在搜索中一直找不到目标节点，而且OPEN表中不再有可供扩展的节点，则搜索失败。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.2 广度优先搜索

- 基本思想：

  从初始节点S_0开始，逐层地对节点进行扩展，并考察它是否为目标节点。在第n层的节点没有全部扩展并考察之前，不对第n＋1层的节点进行扩展。

- OPEN表中节点总是按进入的先后顺序排列，先进入的节点排在前面，后进入的排在后面。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

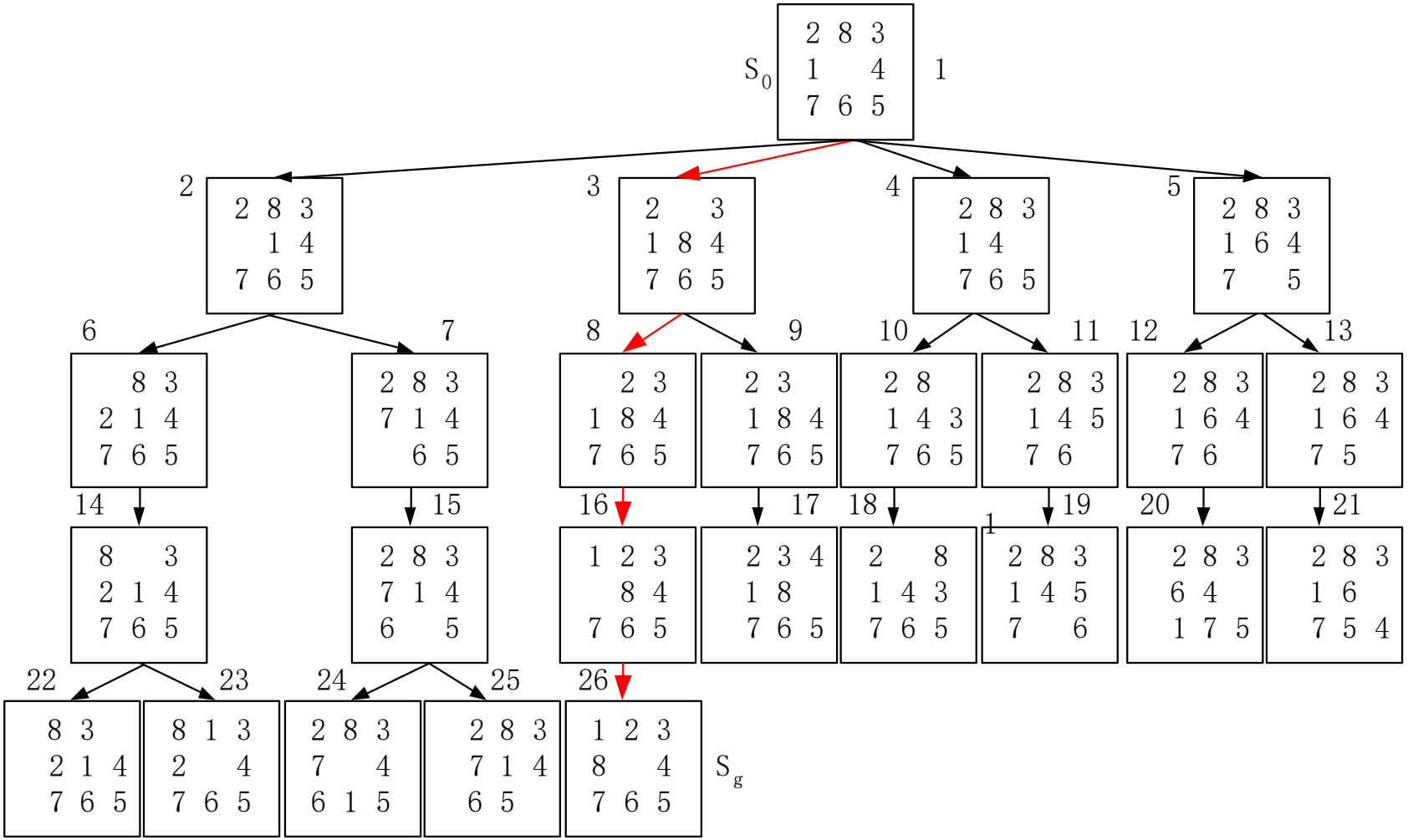
广度优先搜索过程

1. 把初始节点S_0放入OPEN表。
2. 如果OPEN表为空，则问题无解，退出。
3. 把OPEN表的第一个节点取出放入CLOSE表（记为节点n）。
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
5. 若节点n不可扩展，则转第2步。
6. 扩展节点n，将其子节点放入OPEN表的尾部，并为每一个子节点都配置指向父节点的指针，然后转第2步。(只考虑有效子节点，见一般搜索过程)

西安交通大学 XI'AN JIAOTONG UNIVERSITY

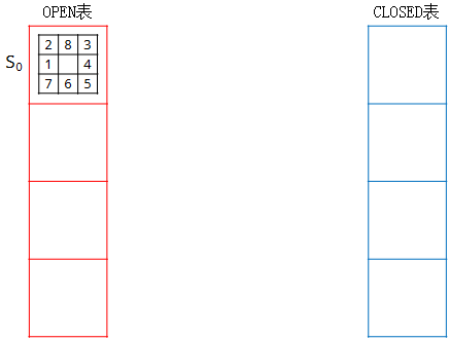
重排九宫的广度优先搜索

操作符：空格左移、上移、右移、下移

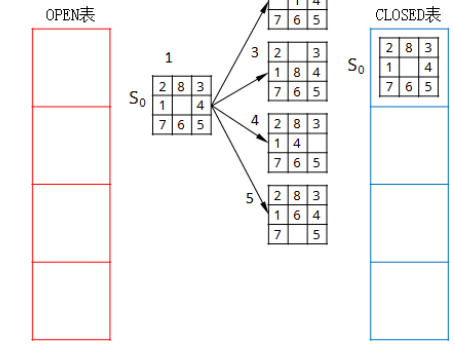


西安交通大学 XI'AN JIAOTONG UNIVERSITY

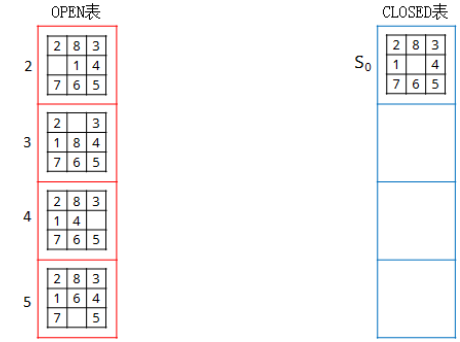
重排九宫的广度优先搜索



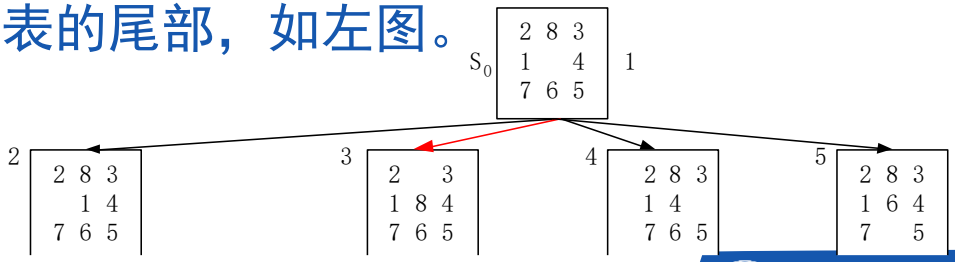
1. 搜索开始，把S_0放入OPEN表。
2. 判断OPEN表是否为空，不是则继续，如左图。



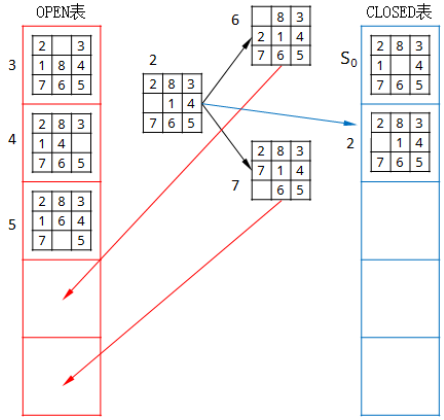
3. 把OPEN表的第一个节点取出放入CLOSE表，判断是否为目标节点，若不是求得该节点的所有可能子节点，如左图。

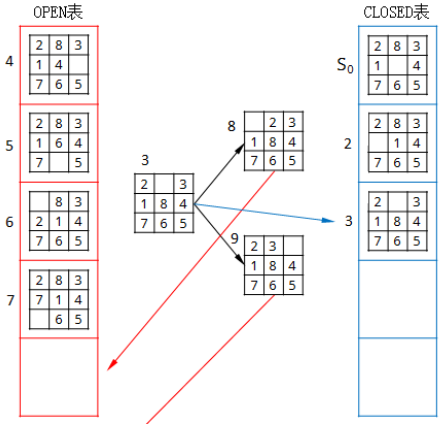


4. 将其中是节点n的先辈的子结点去掉，剩余的放入OPEN表的尾部，如左图。



重排九宫的广度优先搜索





5. 重复2、3、4，判断OPEN表，不为空，继续将OPEN表第一个节点取出并放入CLOSED表中，判断是否为目标节点，若不是求得该节点的所有可能子结点，将其中是节点n的先辈的子结点去掉(这里去掉了一个空格块右移的情况，因为这个情况就是结点2的先辈，也就是结点S_0)，剩余的放入OPEN表的尾部，如左图。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

广度优先搜索

在上述广度优先算法中需要注意几个问题：

1、对于任意一个可扩展的节点，总是按照固定的操作符的顺序对其进行扩展（空格左移、上移、右移、下移）。

2、在对任一节点进行扩展的时候，如果所得的某个子节点（状态）前面已经出现过，则立即将其放弃，不再重复画出（不送入OPEN表）。

3、广度优先搜索的本质是，以初始节点为根节点，在状态空间图中按照广度优先的原则，生成一棵搜索树。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

广度优先搜索的特点

- 优点：

  只要问题有解，用广度优先搜索总可以得到解，而且得到的是路径最短的解（完备性）。

- 缺点：

  广度优先搜索盲目性较大，当目标节点距初始节点较远时将会产生许多无用节点，搜索效率低。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.3 深度优先搜索

- 基本思想：

  从初始节点S_0开始扩展，若没有得到目标节点，则选择最后产生的子节点进行扩展，若还是不能到达目标节点，则再对刚才最后产生的子节点进行扩展，一直如此向下搜索。当到达某个子节点，且该子节点既不是目标节点又不能继续扩展时，才选择其兄弟节点进行考察。

- 深度优先搜索与广度优先搜索的唯一区别是：广度优先搜索是将节点n的子节点放入到OPEN表的尾部，而深度优先搜索是把节点n的子节点放入到OPEN表的首部。

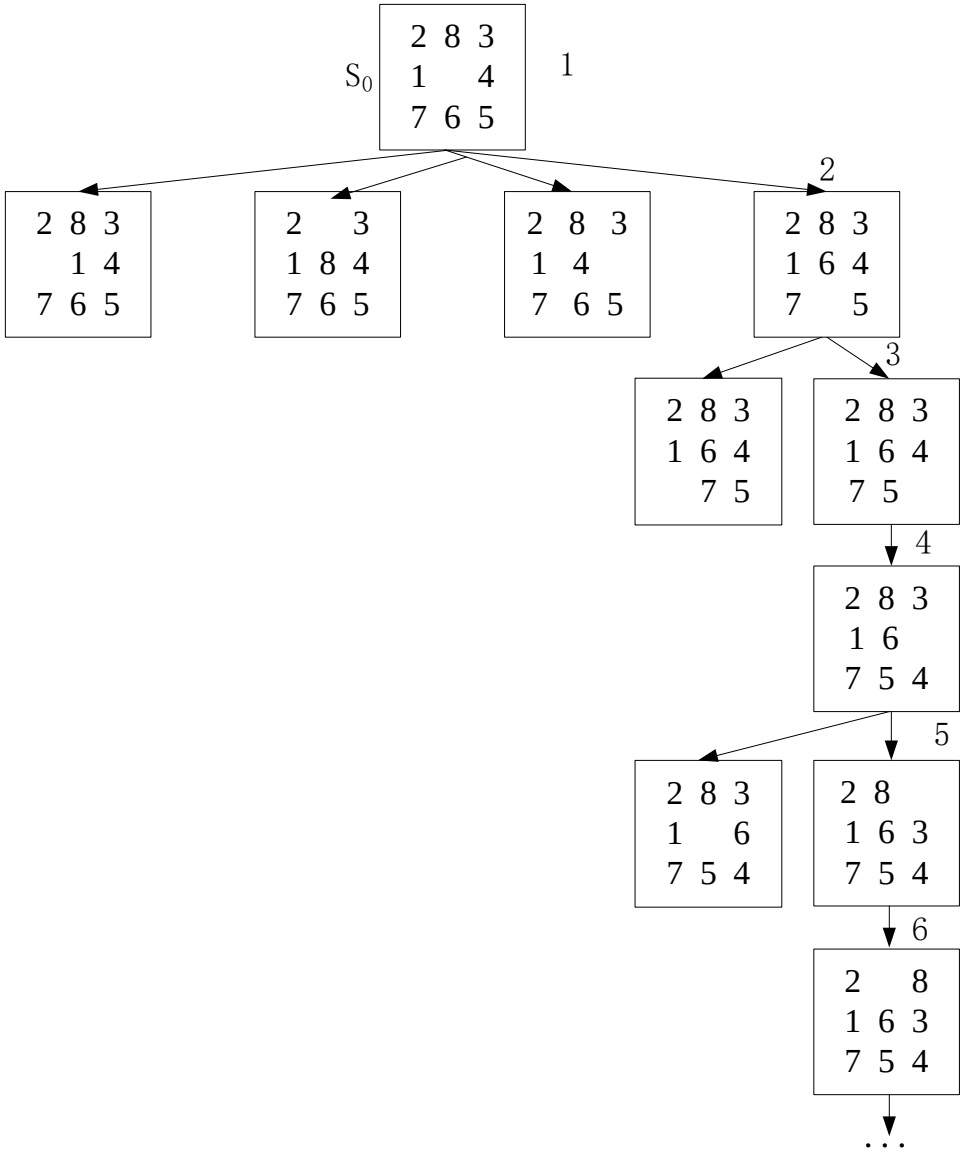
西安交通大学 XI'AN JIAOTONG UNIVERSITY

深度优先搜索过程

1. 把初始节点S_0放入OPEN表。
2. 如果OPEN表为空，则问题无解，退出。
3. 把OPEN表的第一个节点（记为节点n）取出放入CLOSE表。
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
5. 若节点n不可扩展，则转第2步。
6. 扩展节点n，将其子节点放入OPEN表的首部，并为每一个子节点都配置指向父节点的指针，然后转第2步。

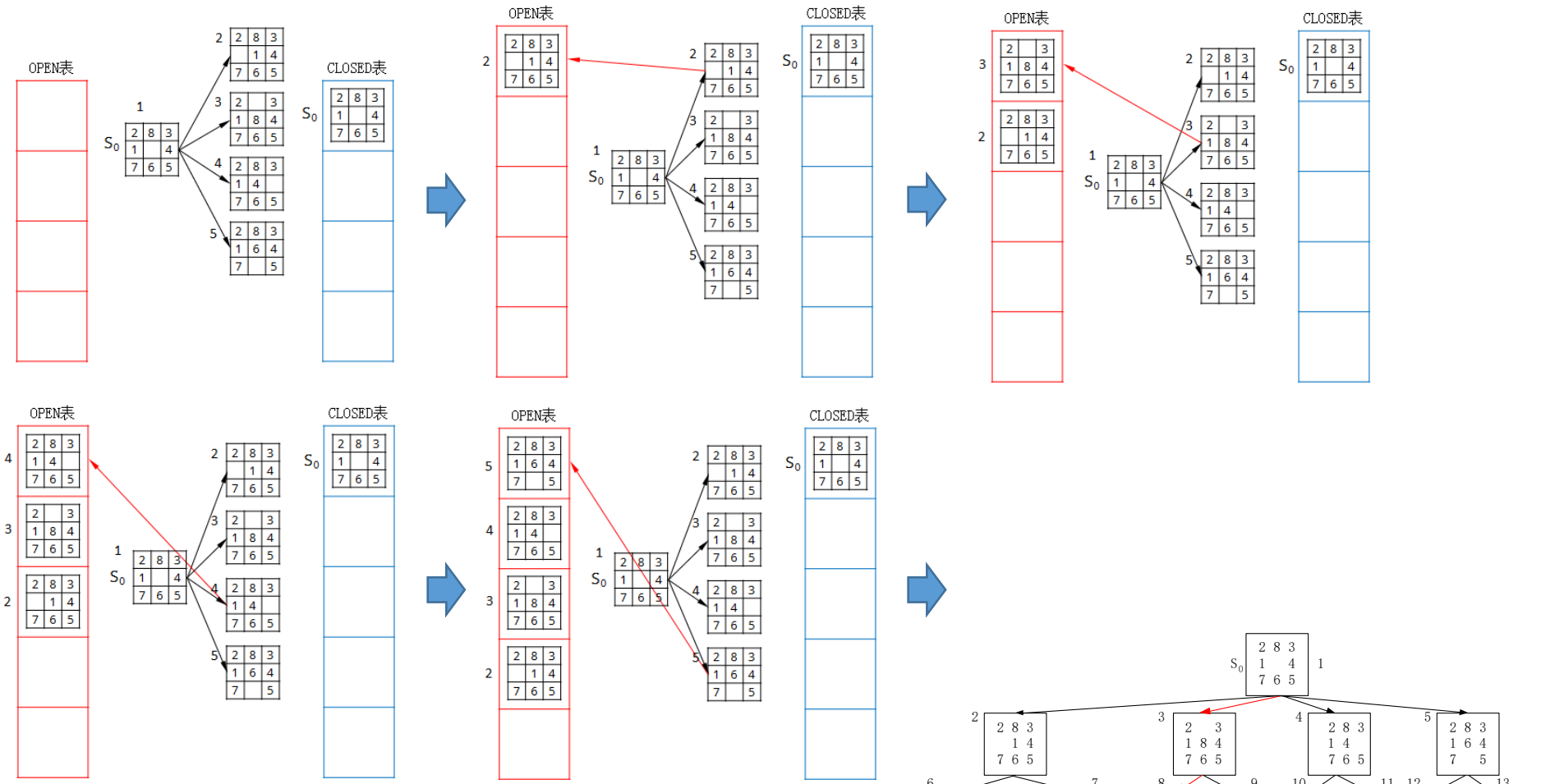
西安交通大学 XI'AN JIAOTONG UNIVERSITY

重排九宫的深度优先搜索

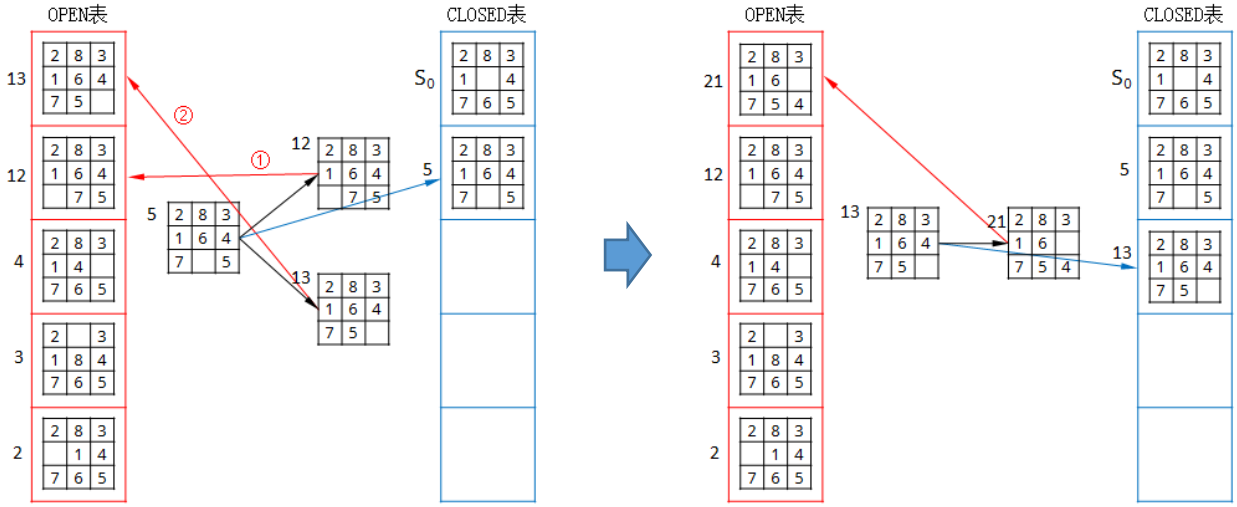


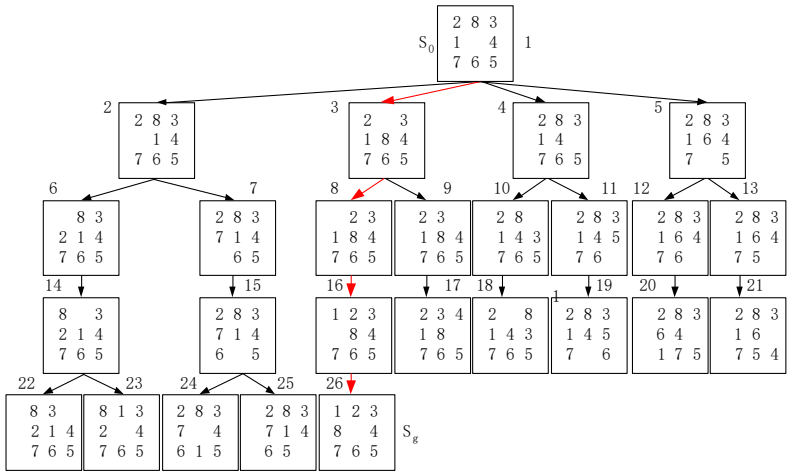
西安交通大學 XI'AN JIAOTONG UNIVERSITY

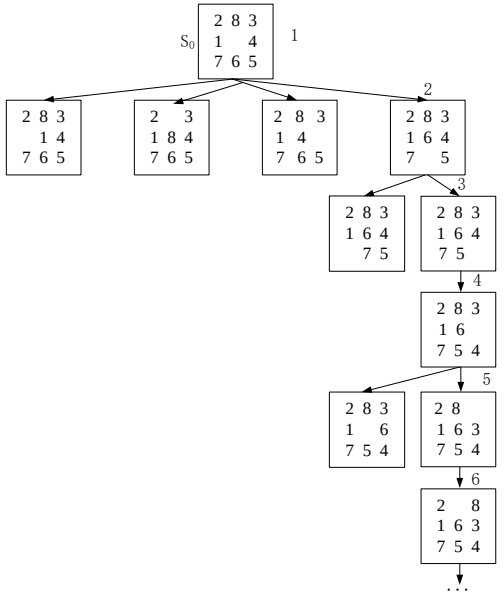
重排九宫的深度优先搜索



重排九宫的深度优先搜索







西安交通大學 XI'AN JIAOTONG UNIVERSITY

深度优先搜索的特点

- 在深度优先搜索中，搜索一旦进入某个分支，就将沿着该分支一直向下搜索。如果目标节点恰好在此分支上，则可较快地得到解。但是，如果目标节点不在此分支上，而该分支又是一个无穷分支，则就不可能得到解。所以深度优先搜索是不完备的，而且即使问题有解，它也不一定能求得解。
- 用深度优先求得的解，不一定是路径最短的解。
- 本质：以初始节点为根节点，在状态空间图中按照深度优先的原则，生成一棵搜索树。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.4 有界深度优先搜索

- 基本思想：
  - 对深度优先搜索引入搜索深度的界限（设为d_m），当搜索深度达到了深度界限，而仍未出现目标节点时，就换一个分支进行搜索。
- 搜索过程：
  1. 把初始节点S_0放入OPEN表中，置S_0的深度$d(S_0)=0$。
  2. 如果OPEN表为空，则问题无解，退出。
  3. 把OPEN表的第一个节点取出放入CLOSE表（记为节点n）。
  4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.4 有界深度优先搜索

5. 若节点n的深度$d(n) = d_m$，则转第2步（此时节点n位于CLOSE表，但并未进行扩展）。
6. 若节点n不可扩展，则转第2步。
7. 扩展节点n，将其子节点放入OPEN表的首部，为每一个子节点都配置指向父节点的指针，将每一个子节点的深度设置为$d(n)$+1，然后转第2步。　　（略）

西安交通大学 XI'AN JIAOTONG UNIVERSITY

有界深度优先搜索的说明

- 如果问题有解，且其路径长度≤ d_m，则上述搜索过程一定能求得解。但是，若解的路径长度> d_m，则上述搜索过程就得不到解。这说明在有界深度优先搜索中，深度界限的选择是很重要的。
- 要恰当地给出d_m的值是比较困难的。
- 即使能求出解，它也不一定是最优解。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

有界深度优先搜索的一些改进方法

1. d_m的选择：

先任意设定一个较小的数作为d_m，然后进行上述的有界深度优先搜索，当搜索达到了指定的深度界限d_m仍未发现目标节点，并且CLOSE表中仍有待扩展节点时，就将这些节点送回OPEN表，同时增大深度界限d_m，继续向下搜索。

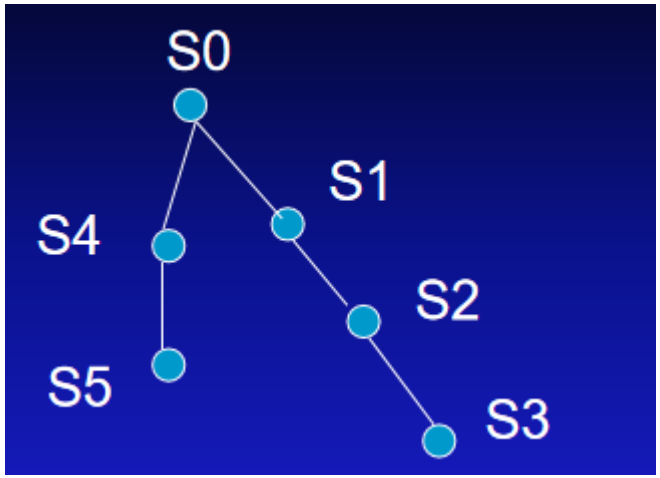
如此不断地增大d_m，只要问题有解，就一定可以找到它。但此时找到的解不一定是最优解。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

有界深度优先搜索的一些改进方法

2. 最优解：

为了找到最优解，可增设一个表R，每找到目标节点S_g后，就把它放入R，并令d_m等于该目标节点所对应的路径长度，然后继续搜索。由于后面求得的解的路径长度不会超过先前求得的解的路径长度，所以后面求得的解一定是最优解。

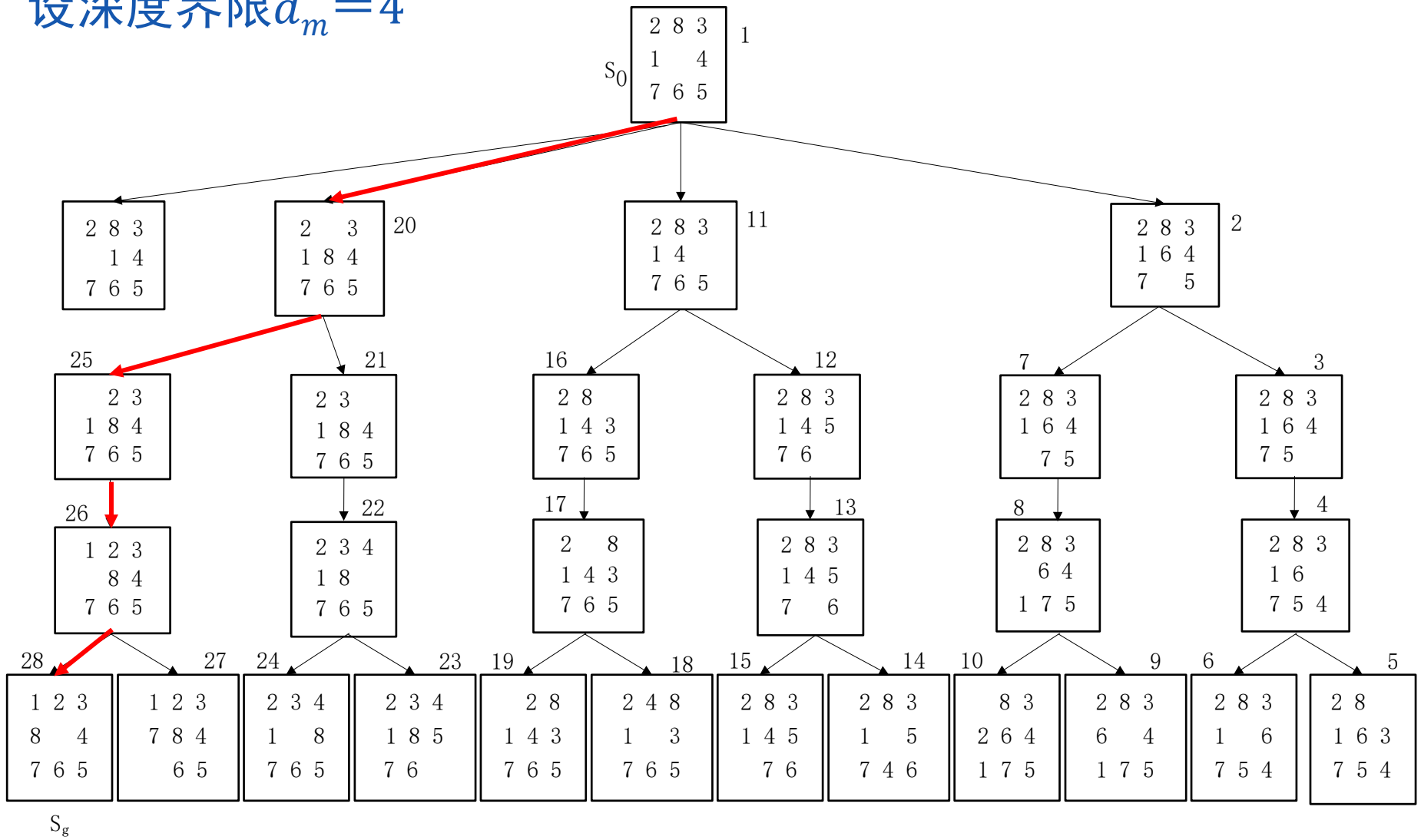


- 例如如果$S3$和$S5$是目标节点，找到$S3$后，取$D_m = 3$，R表中增加$S3$，继续搜索；找到$S5$，取$D_m = 2$，R表中增加$S5$，解为$S5$。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

重排九宫的有界深度优先搜索

设深度界限$d_m=4$



西安交通大學 XI'AN JIAOTONG UNIVERSITY

代价树的广度优先搜索

- 边上标有代价(或费用)的树称为代价树。

- 用$g(x)$表示从初始节点S_0到节点x的代价，用$c(x_1, x_2)$表示从父节点x_1到子节点x_2的代价，则有：

$$g(x_2) = g(x_1) + c(x_1, x_2)$$

西安交通大学 XI'AN JIAOTONG UNIVERSITY

代价树的广度优先搜索

- 基本思想：

  每次从OPEN表中选择节点往CLOSE表传送时，总是选择其中代价最小的节点。也就是说，OPEN表中的节点在任一时刻都是按其代价从小到大排序的。代价小的节点排在前面，代价大的节点排在后面。

- 如果问题有解，代价树的广度优先搜索一定可以求得解，并且求出的是最优解(即该算法是完备的)。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

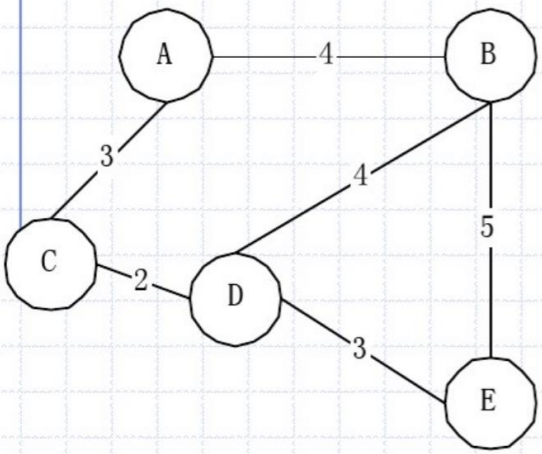
代价树广度优先搜索过程

1. 把初始节点S_0放入OPEN表，令$g(S_0)=0$。
2. 如果OPEN表为空，则问题无解，退出。
3. 把OPEN表的第一个节点（记为节点n）取出放入CLOSE表。
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
5. 若节点n不可扩展，则转第2步。
6. 扩展节点n，为每一个子节点都配置指向父节点的指针，计算各子节点的代价，并将各子节点放入OPEN表中。按各节点的代价对OPEN表中的全部节点进行排序(按从小到大的顺序)，然后转第2步。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

代价树示例

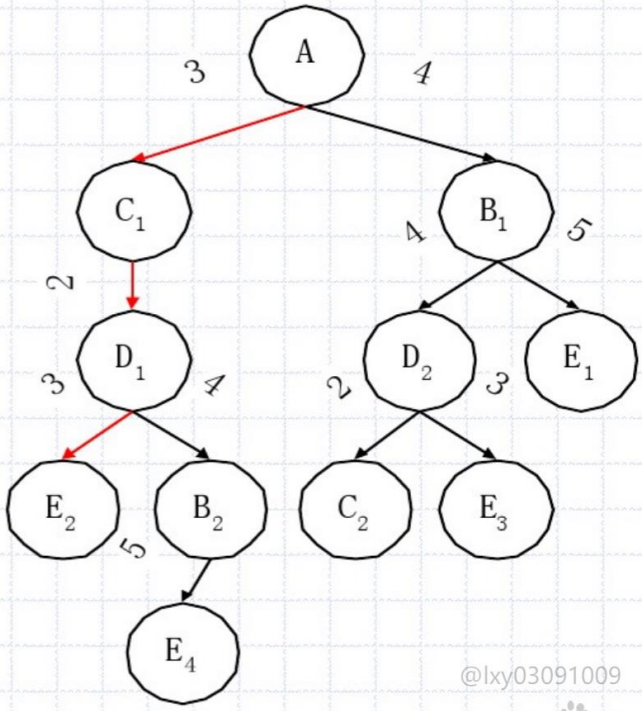
图到代价树的转换



交通图

求A到E的最小费用交通路线

代价树的广度优先搜索



交通图的代价树

@lxy03091009

西安交通大學 XI'AN JIAOTONG UNIVERSITY

代价树的深度优先搜索

- 在代价树的广度优先搜索中，每次都是从OPEN表的全体节点中选择一个代价最小的节点送入CLOSE表进行考察。
- 在代价树的深度优先搜索中，是从刚扩展出的子节点中选择一个代价最小的节点送入CLOSE表进行考察。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

代价树的深度优先搜索

- 搜索过程：
  1. 把初始节点S_0放入OPEN表，令$g(S_0)=0$。
  2. 如果OPEN表为空，则问题无解，退出。
  3. 把OPEN表的第一个节点（记为节点n）取出放入CLOSE表。
  4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
  5. 若节点n不可扩展，则转第2步。
  6. 扩展节点n，将其子节点按代价从小到大的顺序放到OPEN表中的首部，并为每一个子节点都配置指向父节点的指针，然后转第2步。
- 代价树的深度有限搜索是不完备的。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

总结

1、上述各种搜索方法的本质是，以初始节点为根节点，按照既定的策略对状态空间图进行遍历，并希望能够尽早发现目标节点。

2、由于对状态空间图遍历的策略是既定的，因此这些方法统称为盲目搜索方法。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

5.2.5 启发式搜索

- 盲目搜索具有较大的盲目性，产生的无用节点较多，效率不高。
- 启发式搜索采用问题自身的特性信息，以指导搜索朝着最有希望的方向前进。这种搜索针对性较强，因而效率较高。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

1、启发性信息与估价函数

- 可用于指导搜索过程，且与具体问题有关的信息称为启发性信息。
- 用于评估节点重要性的函数称为估价函数。其一般形式为：

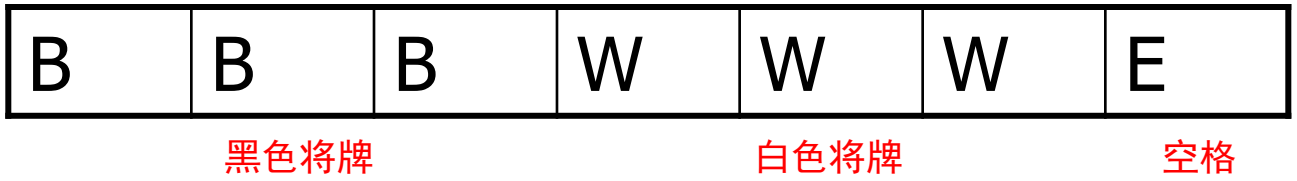
$$f(x) = g(x) + h(x)$$

- 其中$g(x)$表示从初始节点S_0到节点x的代价；$h(x)$是从节点x到目标节点S_g的最优路径的代价的估计，它体现了问题的启发性信息。$h(x)$称为启发函数。
- $g(x)$ 有利于搜索的完备性，但影响搜索的效率。$h(x)$有利于提高搜索的效率，但影响搜索的完备性。确定$f(x)$时要使$g(x)+h(x)$各占适当比重。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

启发函数示例

设有如下结构的移动将牌游戏：



该游戏规则：

1. 当一个牌移入相邻的空位置时，费用为一个单位。
2. 一个牌至多可跳过两个牌进入空位置，其费用等于跳过的牌数加1。

要求：把所有的B都移至W的右边，请设计启发函数$h(x)$。

解：根据要求可知，W左边的B越少越接近目标，因此可用W左边B的个数作为$h(x)$,即

$$h(x) = 3 \times (\text{每个}W\text{左边}B\text{的个数的总和})$$

这里乘以系数3是为了扩大$h(x)$在$f(x)$中的比重。

$$f(x) = g(x) + h(x)$$

启发函数示例

B	E	B	W	W	B	W

$h(x) = 3 \times$ (每个W左边B的个数的总和)

$= 3 \times (2 + 2 + 3) = 21$

西安交通大学 XI'AN JIAOTONG UNIVERSITY

2、局部择优搜索

基本思想：

局部择优搜索是一种启发式搜索方法，是对深度优先搜索方法的一种改进。当一个节点被扩展以后，按$f(x)$对每一个子节点计算估价值，并选择最小者作为下一个要考察的节点。

搜索过程：

1. 把初始节点S_0放入OPEN表，计算$f(S_0)$。
2. 如果OPEN表为空，则问题无解，退出。
3. 把OPEN表的第一个节点（记为节点n）取出放入CLOSE表。
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
5. 若节点n不可扩展，则转第2步。
6. 扩展节点n，用估价函数$f(x)$计算每个子节点的估价值，并按估价值从小到大的顺序放到OPEN表中的首部，并为每一个子节点都配置指向父节点的指针，然后转第2步。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

2、局部择优搜索

- 在局部择优搜索中，若令$f(x) = g(x)$，则局部择优搜索就成为代价树的深度优先搜索。
- 在局部择优搜索中，若令$f(x) = d(x)$，这里$d(x)$表示节点x的深度，则局部择优搜索就成为深度优先搜索。

因此，深度优先搜索、代价树的深度优先搜索均为局部择优搜索的特例。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

3、全局择优搜索

- 基本思想：

  每当要选择下一个节点进行考察时，全局择优搜索每次总是从OPEN表的全体节点中选择一个估价值最小的节点。

- 搜索过程：

1. 把初始节点S_0放入OPEN表，计算$f(S_0)$。　　$f(x) = g(x) + h(x)$
2. 如果OPEN表为空，则问题无解，退出。
3. 把OPEN表的第一个节点（记为节点n）取出放入CLOSE表。
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出。
5. 若节点n不可扩展，则转第2步。
6. 扩展节点n，根据估价函数$f(x)$计算每个子节点的估价值，并为每一个子节点都配置指向父节点的指针。把这些子节点都送入OPEN表中，然后对OPEN表中的全部节点按估价值从小至大的顺序进行排序，然后转第2步。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

3、全局择优搜索

- 在全局择优搜索中，若令$f(x)=g(x)$，则全局择优搜索就成为代价树的广度优先搜索。
- 在全局择优搜索中，若令$f(x)=d(x)$，这里$d(x)$表示节点x的深度，则全局择优搜索就成为广度优先搜索。

因此，广度优先搜索、代价树的广度优先搜索是全局择优搜索的特例。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

重排九宫问题的全局择优搜索树

- 设估价函数为：$f(x)=g(x)+h(x)=d(x)+h(x)$，其中$d(x)$表示节点x的深度，起始节点S_0的深度为0，$h(x)$表示节点x的格局与目标节点格局不相同的牌数（“不在位”的数字个数）。

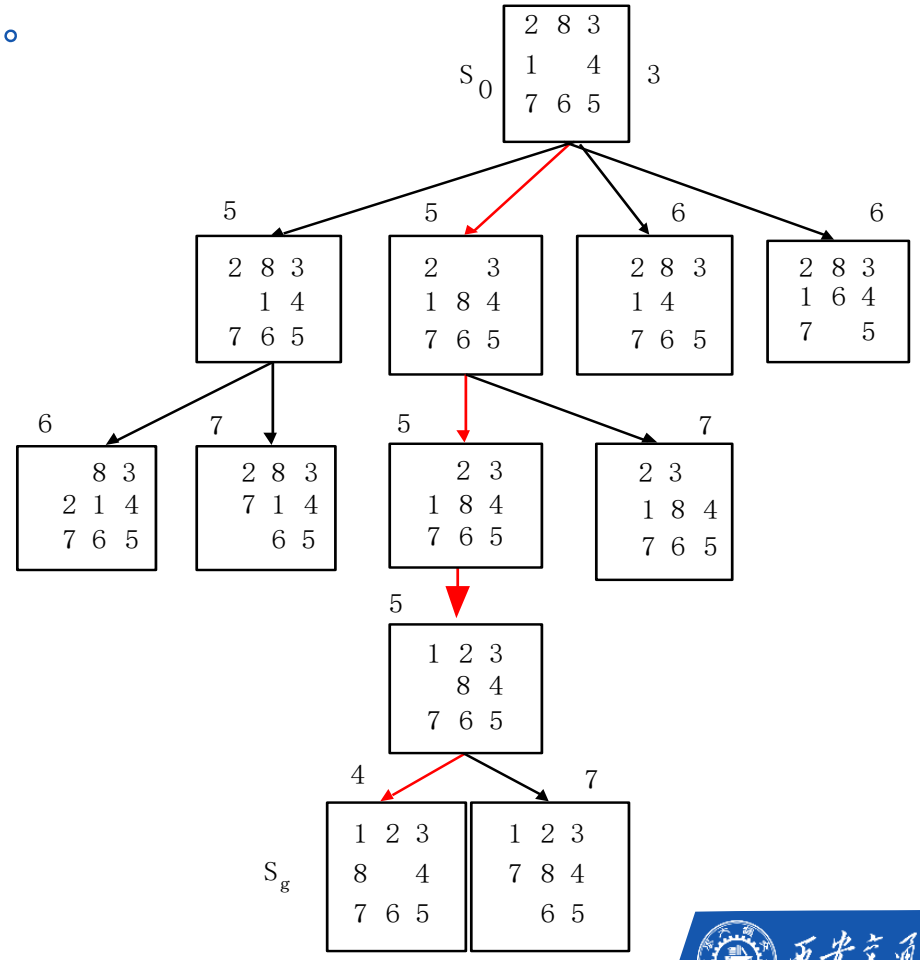
S_0

2	8	3
1		4
7	6	5

Sg

1	2	3
8		4
7	6	5

$$h(S_0)=3$$



5.2.6 A*算法

- 如果5.2.1中给出的一般搜索过程满足如下限制，则它就称为A*算法：

1、把OPEN表中的节点按估价函数

$$f(x) = g(x) + h(x)$$

的值从小至大进行排序（一般搜索过程的第7步）。

2、$g(x)$是从初始节点S_0到节点x的路径的实际代价，$g(x)$是对$g^*(x)$的估计，$g^*(x)$是从初始S_0到节点x的实际最小代价。

3、$h(x)$是$h^*(x)$的下界，即对所有的x均有：

$$h(x) \leq h^*(x)$$

其中，$h^*(x)$是从节点x到目标节点的实际最小代价。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

A*算法启发函数

- $h(x)=0$：由$g(x)$决定节点的优先级，此时算法就退化成广度优先算法。
- $h(x) \leq h^*(x)$：A*算法保证一定能够找到最短路径。但是当$h(x)$的值越小，算法将遍历越多的节点，也就导致算法越慢。
- $h(x)=h^*(x)$：A*算法将找到最佳路径，并且速度很快。可惜的是，并非所有场景下都能做到这一点。因为在没有达到终点之前，很难确切算出距离终点还有多远。
- $h(x)>h^*(x)$：A*算法不能保证找到最短路径，不过此时会很快。
- $h(x)>>g(x)$：此时只有$h(x)$产生效果，这也就变成了最佳优先搜索。

➢ 由上可知：通过调节启发函数可以控制算法的速度和精确度。因为在一些情况，可能未必需要最短路径，而是希望能够尽快找到一个路径即可。这也是A*算法比较灵活的地方。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

A*算法搜索过程

1. 把初始节点S_0放入OPEN表，并建立目前只包含S_0的图，记为G；
2. 检查OPEN表是否为空，若为空则问题无解，退出；
3. 把OPEN表的第一个节点取出放入CLOSE表，并计该节点为n；
4. 考察节点n是否为目标节点。若是，则求得了问题的解，退出；
5. 扩展节点n，生成一组子节点。把其中不是节点n先辈的那些子节点记做集合M，并把这些子节点作为节点n的子节点加入G中；
6. 针对M中子节点的不同情况，分别进行如下处理：
   ① 对于那些未曾在G中出现过的M成员设置一个指向父节点（即节点n）的指针，并把它们放入OPEN表；（不在OPEN表）
   ② 对于那些先前已经在G中出现过的M成员，确定是否需要修改它指向父节点的指针；（在OPEN表,对$g(x)$进行更新）
   ③ 对于那些先前已在G中出现并且已经扩展了的M成员，确定是否需要修改其后继节点指向父节点的指针；（在CLOSE表中, 对节点n子节点的子节点更新$g(x)$）
7. 对OPEN表中的节点按估价函数$f(x) = g(x) + h(x)$进行排序；
8. 转第2步。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

A*算法

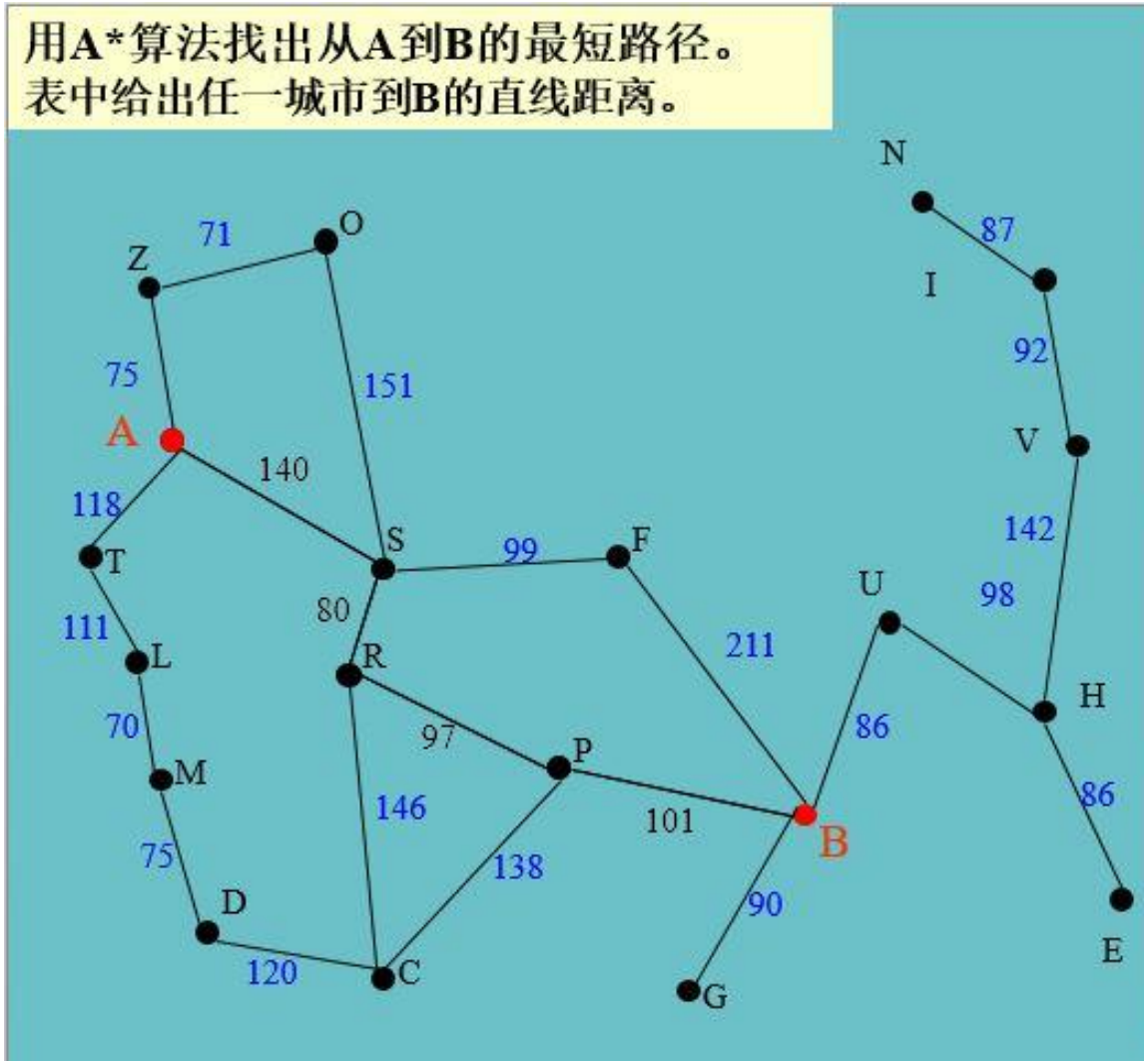
- 在A*算法中，$g(x)$ 是从初始节点S_0到节点x的路径的代价，恒有$g(x) \geq g^*(x)$。而且在算法执行过程中随着更多搜索信息的获得，$g(x)$的值呈下降的趋势。
- $h(x)$的设计依赖于具体问题领域的启发性信息。$h(x) \leq h^*(x)$限制了启发式信息在估价函数中的比重，有利于算法的稳定性，保证了A*算法总能找到最优解。

$$f(x) = g(x) + h(x)$$

西安交通大学 XI'AN JIAOTONG UNIVERSITY

A*算法举例

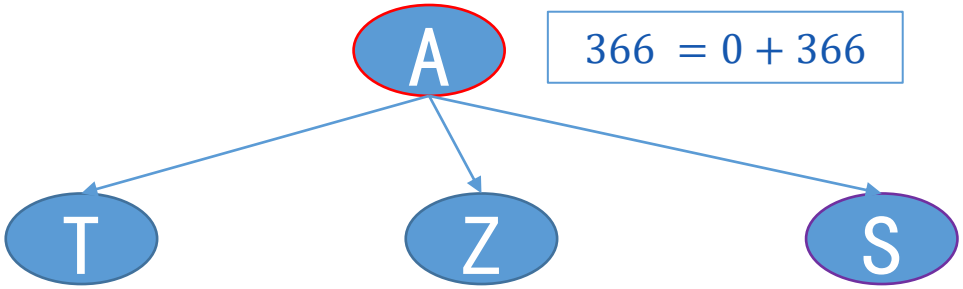
用A*算法找出从A到B的最短路径。
表中给出任一城市到B的直线距离。



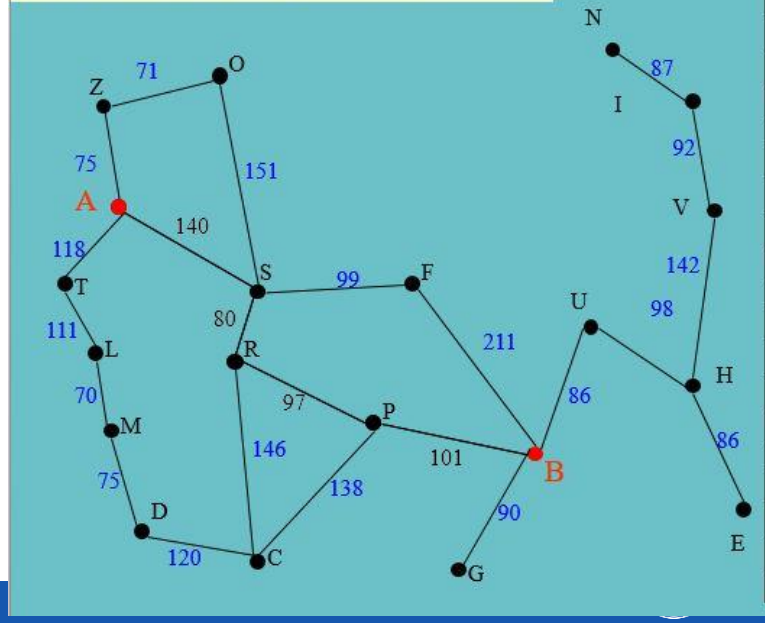
城市	直线距离
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

西安交通大學 XI'AN JIAOTONG UNIVERSITY

A*算法举例

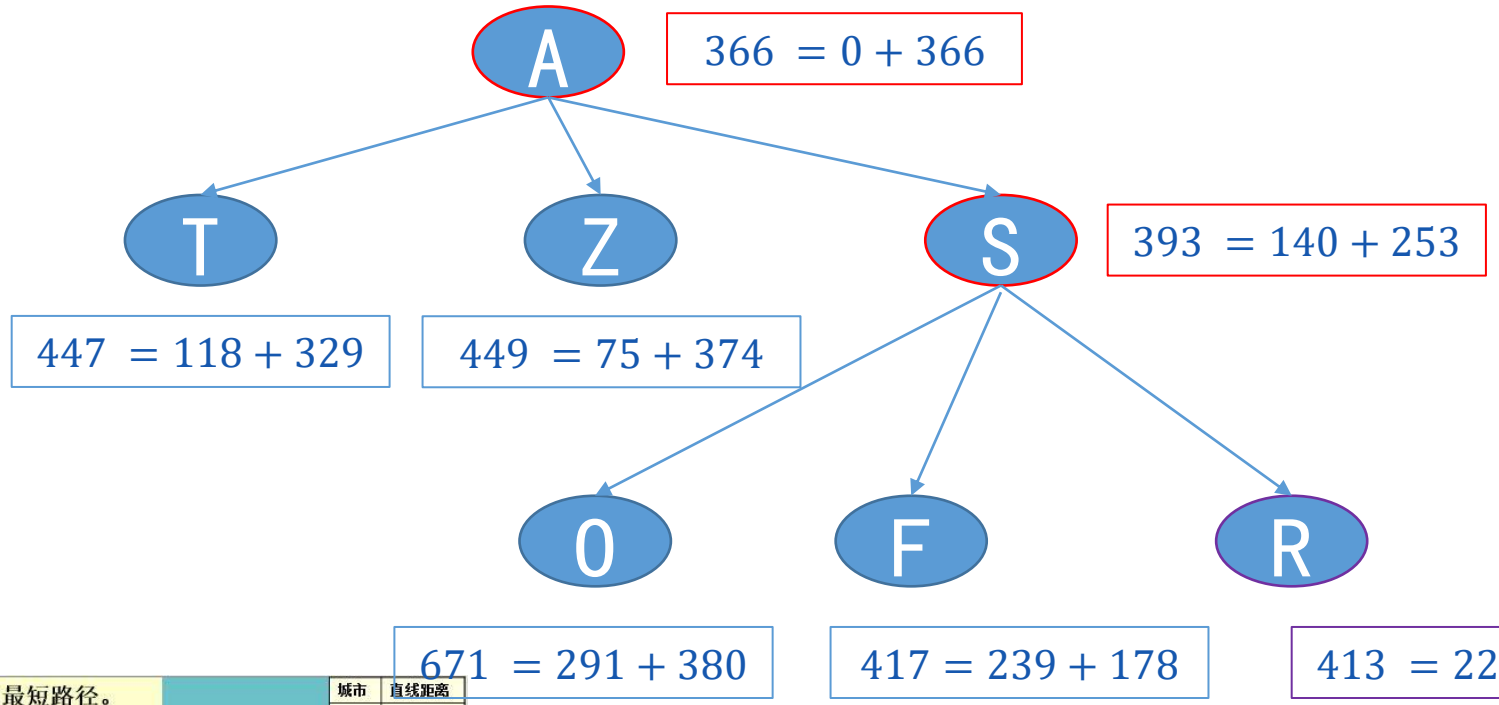


用A*算法找出从A到B的最短路径。
表中给出任一城市到B的直线距离。

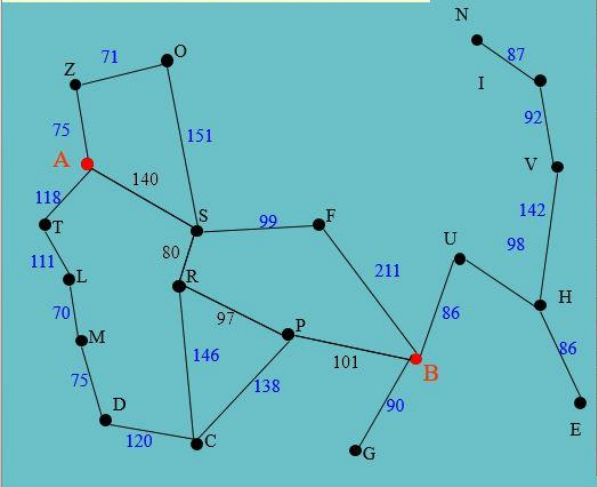


城市	直线距离
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

A*算法举例



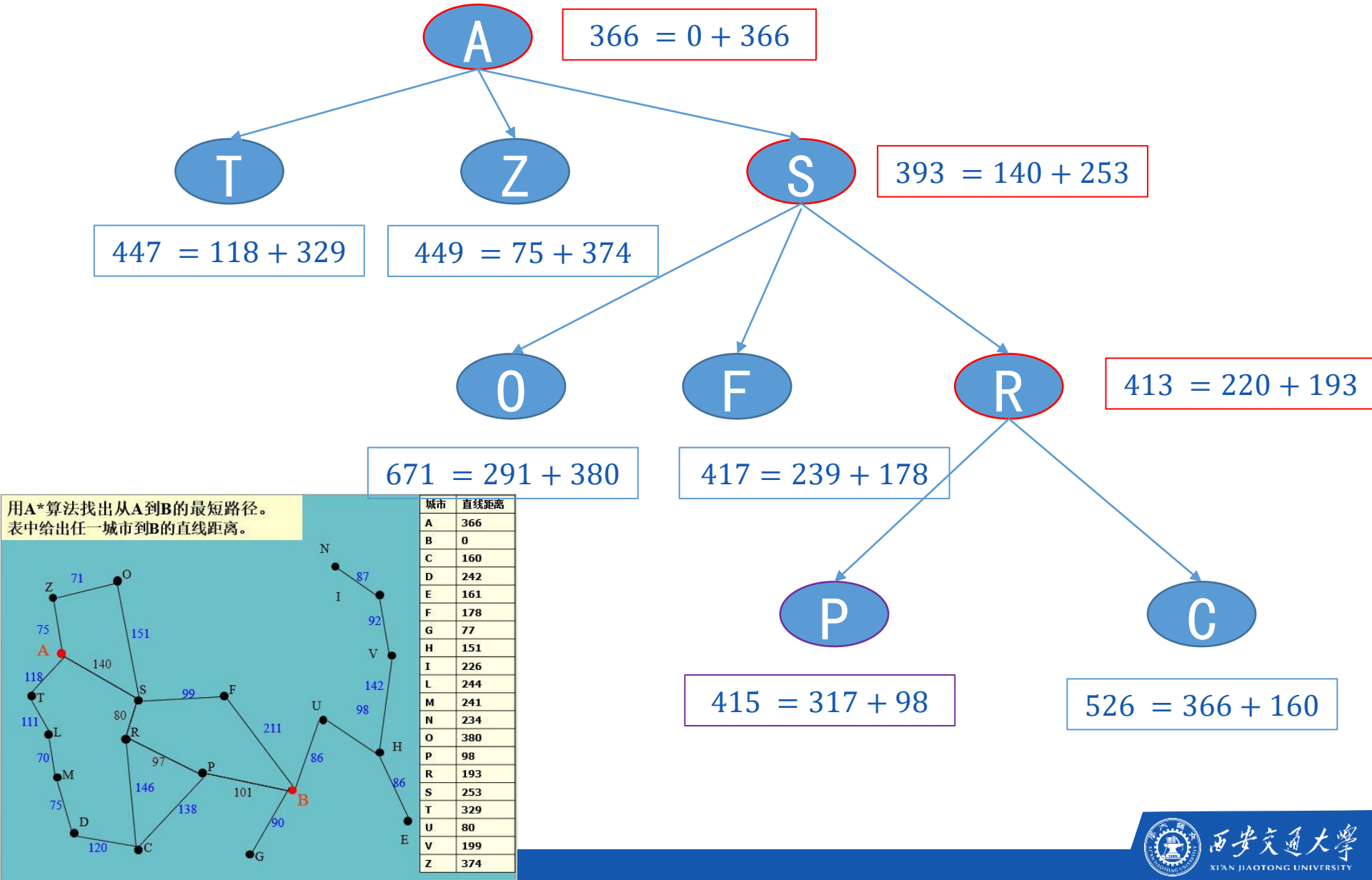
用A*算法找出从A到B的最短路径。
表中给出任一城市到B的直线距离。



城市	直线距离
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

西安交通大學 XI'AN JIAOTONG UNIVERSITY

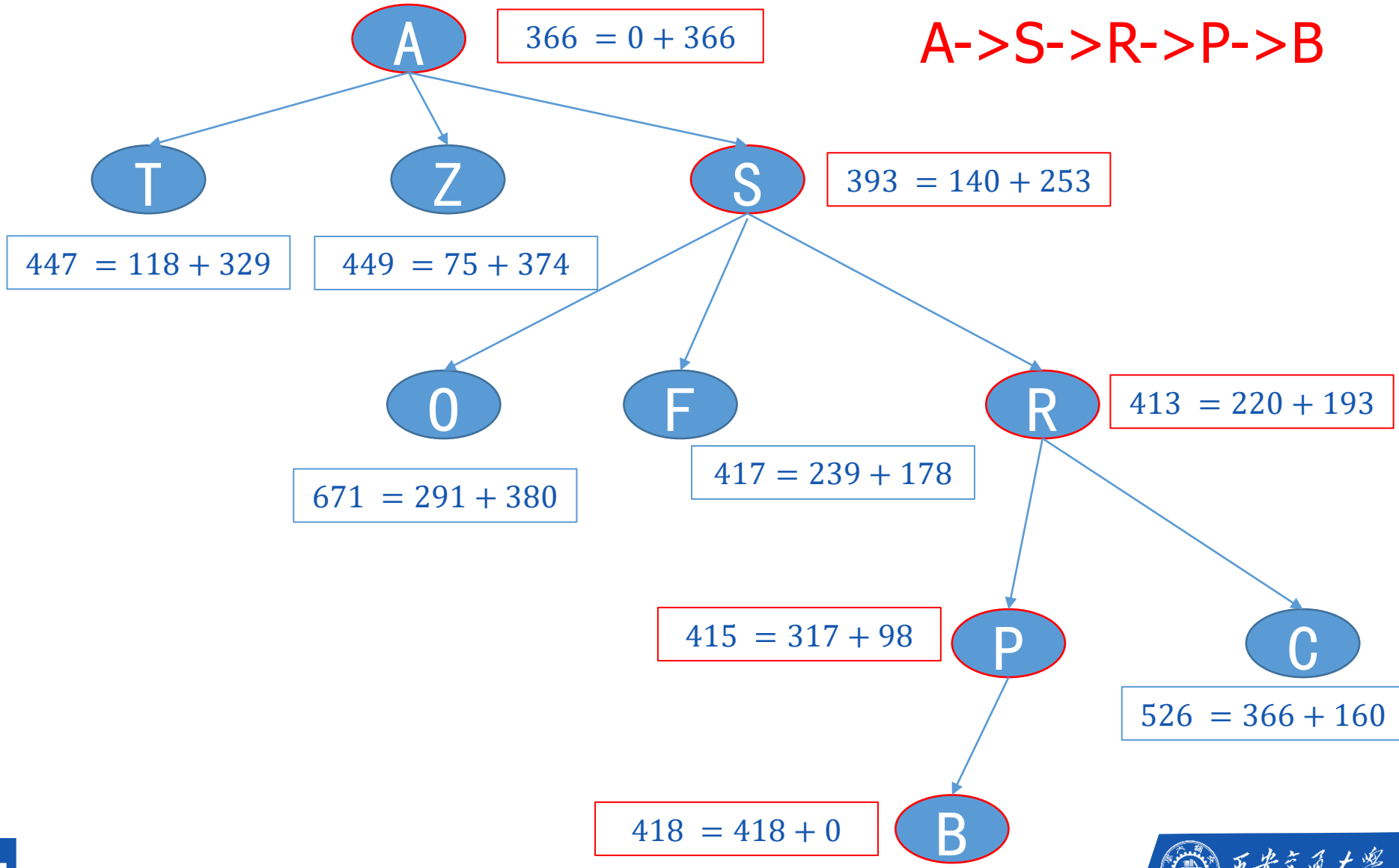
A*算法举例



用A*算法找出从A到B的最短路径。
表中给出任一城市到B的直线距离。

城市	直线距离
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

A*算法举例



西安交通大学 XI'AN JIAOTONG UNIVERSITY

1、A*算法的可纳性

- A*算法的可纳性

  对于可解状态空间图(即从初始节点到目标节点有路径存在)，如果一个搜索算法能在有限步内终止，并且能找到最优解，则称该搜索算法是可纳的。A*算法是可纳的。

证明步骤：

（1）对于有限图，A*算法一定会在有限步内终止。

（2）对于无限图，只要从初始节点到目标节点有路径存在，则A*算法也必然会终止。

（3）A*算法一定终止在最优路径上。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

2、A*算法的最优性

- A*算法的搜索效率在很大程度上取决于$h(x)$，在满足$h(x) \le h^*(x)$的条件下，$h(x)$的值越大越好。$h(x)$的值越大，表明它携带的启发性信息越多，搜索时扩展的节点数越少，搜索的效率越高。
- 若有两个启发函数$h_1(x) \le h^*(x)$与$h_2(x) \le h^*(x)$，且有$h_1(x) \le h_2(x)$，那么$h_2(x)$扩展的节点集合是$h_1(x)$扩展节点集合的子集。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

3、h(x)的单调性

- 搜索节点数目少≠搜索速度快
- 在A*算法中，每当扩展一个节点(节点n)时都要检查其子节点是否在OPEN表或CLOSE表中，如果在，则要调整子节点指向父节点的指针，这就增加了搜索的代价。如果对启发函数$h(x)$加上单调性限制，就可免除大部分上述检查及调整工作，从而减少搜索代价。

西安交通大学 XI'AN JIAOTONG UNIVERSITY

3、h(x)的单调性

- 所谓单调性限制是指$h(x)$满足如下两个条件：

1、$h(S_g) = 0$;

2、设x_j是节点x_i的子节点，则有

$$h(x_i) \leq h(x_j) + c(x_i, x_j)$$

其中，S_g是目标节点；$c(x_i, x_j)$是节点x_i到其子节点x_j的代价。

- 可以证明，当A*算法的启发函数$h(x)$满足单调性限制时，可得到如下两个结论：

1、若A*算法选择节点x_n进行扩展，则

$$g(x_n) = g^*(x_n)$$ (由于该性质故可不作有些子节点指针的更新)

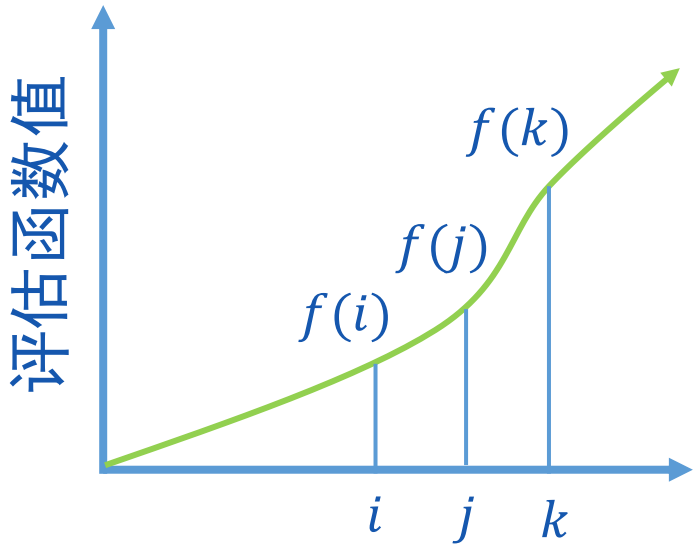
2、由A*算法所扩展的节点序列其f值是非递减的（递增的）。

$$f(x) = g^*(x) + h(x)$$

3、h(x)的单调性

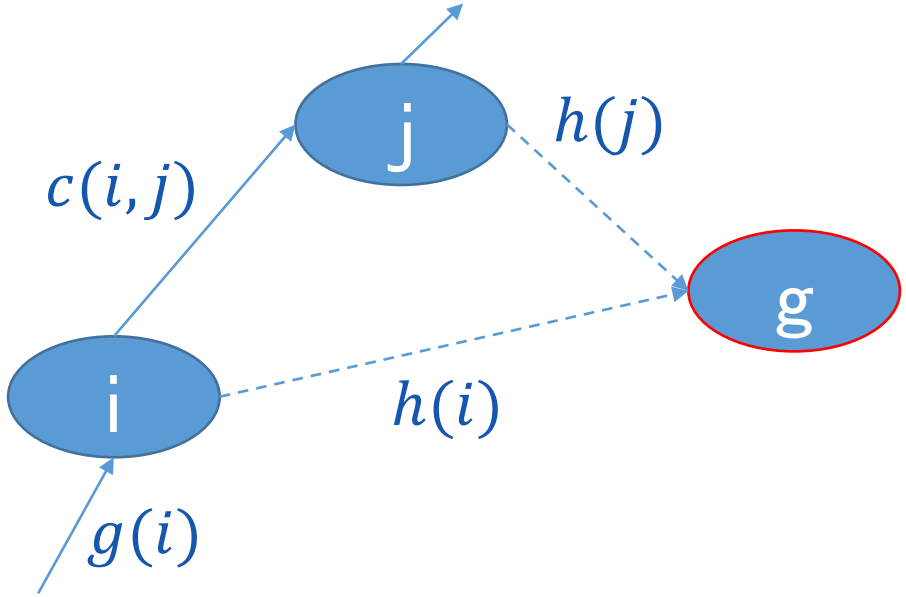
- 评估函数的单调性：节点对$(i，j)$，j是i的后继节点（子节点），如果评估函数f对于所有的这种节点对都满足：

$$f(i) \leq f(j)$$



扩展顺序的先后

3、h(x)的单调性



$$f(i) \leq f(j)$$

$$g(i) + h(i) \leq g(j) + h(j),\ g(j) = g(i) + c(i,j)$$

$$h(i) \leq c(i,j) + h(j)$$

即评估函数单调性约束下启发函数应该满足的条件

完

谢谢